Egg segmentation and tracking: Nosh Robotics CV intern task

Kshitij Kumar Jha. Code: https://github.com/KKJ10000/nosh-egg-segmentation. Tracking video (public): https://drive.google.com/file/d/10-KmLy1rw8uT85Oxb9gxMyuJAdiJGkqu/view?usp=sharing

Summary

- **Task 1 (pretrained):** SAM ViT-B automatic masks plus a shape/brightness egg filter give 144 egg instances on the 110 images. These masks are the pseudo ground truth for Tasks 2-3.
- **Task 2 (classical):** a leakage-aware 4:1 split by scene (88 dev / 22 eval images), then 32 logged trials on dev only. The final pipeline (trial T31) is a locally normalised Lab whiteness map, a multi-level threshold sweep plus Canny closed-contour candidates, morphology, distance-transform watershed splitting of non-elliptical blobs, ellipse-shape and contrast scoring, and marker-controlled watershed refinement of each egg's edge. Dev mean IoU went from 0.327 (gray + Otsu baseline, T01) to 0.583.
- **Task 3 (evaluation, held-out eval split, 22 images):** mean IoU **0.543**, mean Dice **0.581**, pixel precision 0.573, pixel recall 0.599, boundary F **0.502**, pooled IoU 0.604, instance F1 0.588, exact egg-count accuracy 0.545 (count MAE 0.59). Results are bimodal: 14 of 22 eval images reach IoU 0.52-0.99 (9 are at 0.85 or above), and 8 are complete misses (IoU 0), mostly white eggs on white or bright textured backgrounds.
- **Task 4 (tracking):** classical per-frame segmentation, Kalman filter + Hungarian matching and lost-track re-identification keep exactly 2 IDs for the 2 eggs over the whole 60 s video. The video was produced with an earlier version of the segmenter (see Task 4).

Data

The dataset is the Roboflow "egg" v1 export from the assignment's zip: 110 JPEGs at 640x640, exported in *folder* (classification) format. The only labels are folder names (egg / Unlabeled), so there are **no boxes or masks**. The images are mostly one to three white eggs, photographed by phone on very varied backgrounds: sand and ash, steel bowls with water, pink plastic bowls, green leaves, keyboards, towels, egg trays, foil packets and hands. Many backgrounds are themselves white or bright, which is what makes classical segmentation hard here.

All work was done on a 2-core CPU with 7 GB of RAM and no GPU.

Task 1: Segmentation with a pretrained model (SAM)

Method

- **Model:** Segment Anything, ViT-B checkpoint (`sam_vit_b_01ec64.pth`), run on CPU. The image encoder alone takes about 16 s per image at SAM's 1024 px input.
- **Prompting:** with no boxes in the dataset, box prompts were not possible. I used SAM's automatic mask generator with a 10x10 point grid (`pred_iou_thresh=0.86`, `stability_score_thresh=0.90`, `min_mask_region_area=200`). That took about 30–45 s per image, about 70 min for all 110. A 16x16 grid took about 60 s per image, which did not fit the time budget.
- **Caching:** every raw proposal is cached (`outputs/sam_raw/*.npz`), so the egg filter can be revised without re-running SAM.
- **Egg filter (`src/sam_filter.py`):** SAM proposes everything (bowls, leaves, keyboard keys, highlights). A proposal is kept as an egg if: - its area is 0.4–60% of the image; - it touches at most one image border, which rejects wedge-shaped corner fragments; - its solidity is at least 0.93; - its IoU with its own least-squares ellipse is at least 0.88; - its minor/major axis ratio is at least 0.45; - its mean Lab L is at least 120, it is not darker than a 31 px ring around it, and its mean chroma is at most 40.
- **De-duplication:** survivors are processed largest first, and a proposal is dropped if more than 40% of it is already covered. This keeps the whole egg rather than SAM's sub-part masks (highlight, shadow half).
- **Fallback:** if nothing passes, a relaxed pass (solidity 0.88, ellipse IoU 0.80, L 100) keeps the single brightest candidate.
- **Output:** `outputs/sam_masks/<image>.png` (uint16 instance-id map, 0 = background), plus a montage of all 110 images.

Result: the filter kept 144 egg instances on the 110 images: 81 images with one egg, 10 with two, 13 with three, 1 with four, and 5 with no accepted egg (all 5 in dev). The eval split has 23 eggs in 22 images (21 single-egg images and one two-egg image); dev has 121 eggs in 88 images.



SAM ViT-B + egg filter on all 110 images (each colour is one instance). These masks are the pseudo ground truth for Tasks 2-3.

Caveats about the pseudo-GT

The egg filter uses shape and brightness priors similar to the ones the classical method uses. Because of that, the GT is biased toward bright, elliptical, mostly unoccluded eggs: - Eggs that are partly occluded (in a plastic bag, among leaves, under water) are sometimes missed by the filter or only partly masked. - In a few images the fallback picked the wrong region.

These GT errors are counted against the classical method in Task 3 and are pointed out in the failure analysis where they occur. With more time I would prompt SAM with a single positive point at each egg (one click per egg) to get a clean GT.

Task 2: Classical image-processing segmentation

Dev/eval split with leakage reduction (4:1)

The export contains no Roboflow augmentation copies: all 110 original filenames (`not_damaged_<n>`) are unique, and only one pair is a near-duplicate (pHash distance 4). The real leakage risk is **same-scene photos**. The same egg on the same ash heap, steel bowl, pink bowl, keyboard or foil packet was photographed several times from slightly different angles. If such shots are split across dev and eval, eval effectively measures memorised backgrounds.

`src/split.py` therefore groups images before splitting: 1. **Background similarity:** an HSV colour histogram (12x6x6 bins) per image, Bhattacharyya distance, and complete-linkage clustering cut at 0.55. 2. **Near-duplicates:** images with pHash Hamming distance of 12 or less are merged into the same group. 3. **Whole groups go to one split:** groups are assigned to eval (seed 0) until it holds 20% of the images, preferring groups of 6 or fewer so eval keeps scene diversity.

I chose the cut of 0.55 by looking at montages of the groups. At 0.35, same-scene shots were still split apart (76 groups). At 0.55 the groups correspond to scenes (38 groups). After the split, the closest dev–eval pair has a pHash distance of 16, against 4 for the closest pair overall. The eval split ends up with keyboard, towel, hand, foil-packet, door and pink-bowl scenes; the ash, steel-bowl and egg-tray scenes are in dev.

Item	Value
Images	110
Scene groups	38 (sizes [9, 7, 6, 5, 5, 5, 5, 4, 4, 4] ...)
Dev / Eval	88 / 22
Grouping params	{"phash_t": 12, "hist_t": 0.55, "eval_frac": 0.2, "seed": 0}

I visually checked all 22 eval pseudo-GT masks: every egg is correctly segmented (21 single-egg images and one two-egg image). Dev contains about 9 images whose SAM masks are wrong or empty (eggs under water in steel bowls, eggs among leaves, backlit eggs, the egg tray). Those images lower the dev numbers for every trial equally, so they do not change which trial wins.

Final algorithm

`src/classical.py` with `results/final_config.json`. No learning is involved; every step is a hand-designed operation:

- **Feature map:** Lab whiteness `L - 1.5*chroma`, minus 0.5x a Gaussian-blurred copy (sigma 60 px) to remove the local background brightness, blurred (5 px) and normalised to 0-255.
- **Candidates:** a sweep of 12 thresholds at the 40th-97th percentiles of the feature map. Each binary map is opened and closed with a 7 px elliptical kernel and hole-filled, and every connected component is a candidate. Closed contours from an auto-threshold Canny edge map are added as further candidates.
- **Splitting merged blobs:** a candidate that fails the shape test is split with a distance-transform watershed (markers at regional maxima above 0.6x the maximum distance), and each part is tested again. This is the T31 change.
- **Shape test:** area 0.4-60% of the image, solidity of at least 0.90, IoU with the fitted ellipse of at least 0.85, axis ratio of at least 0.4.
- **Scoring and selection:** score = ellipse IoU x solidity x min(1, ring contrast / 40). Candidates scoring below 0.85x the best in the image are dropped, then a greedy non-maximum suppression (overlap 0.3) keeps the best non-overlapping candidates.
- **Edge refinement:** each kept egg is snapped to the colour edge with a marker-controlled watershed: sure foreground is the candidate eroded by 0.35r, sure background is outside the candidate dilated by 0.7r (r = equivalent radius).
- **Speed:** about 1.8 s per image on the 2-core CPU (2 worker processes).

Experiment log (dev split only, 88 images, SAM pseudo-GT)

Every row below is one real run of `src/run_trial.py`, exactly as logged in `results/experiment_log.csv`. The exact commands are in `scripts/trials.sh`. Configs are overrides of `classical.BASE`. **IoU, Dice, precision, recall and BF are per-image means.** Inst F1 is instance matching at IoU 0.5. s/img is wall time on the 2-core box (2

processes; CPU steal was high at times). The decision column was written after looking at each result.

Trials were run in stages, and each stage was designed from the error maps of the previous best. Stage A built the basic blocks. Stage B changed how candidates are scored and selected. Stage C combined winners and added boundary refinement. Stage D targeted the 23 dev images with IoU below 0.1. Stage E combined and tuned. Stage F did the final tuning, and Stage G made the last checks around the best configuration.

Trial	What changed and why	Params (overrides vs previous / base)	IoU	Dice	Prec	Rec	BF	Inst F1	Count MAE	s/img	Decision
T01	Baseline: grayscale, global Otsu, keep every connected component. Eggs are bright, so a single global threshold is the simplest possible detector.	{}	0.327	0.438	0.360	0.887	0.235	0.035	26.51	0.25	Baseline. High recall (0.89) but precision 0.36: whole bright background regions are segmented and count error is huge (~26 blobs/img).
T02	Keep only the largest component, since most images contain one egg; tests whether fragment false positives dominate the error.	{"keep": "largest"}	0.306	0.398	0.360	0.740	0.250	0.191	0.49	0.19	Discarded: IoU 0.327 -> 0.306. Largest blob is often bright background (white cloth/paper), not the egg.
T03	Lab L channel instead of gray: perceptual lightness, less affected by colour casts.	{"feature": "L"}	0.306	0.418	0.335	0.906	0.219	0.032	28.08	0.22	Discarded: IoU 0.306 (< gray 0.327). Lab L alone no better than gray.
T04	Whiteness map L - 1.5*chroma: eggs are bright AND colourless, so penalising chroma should suppress bright coloured backgrounds (pink bowls, green leaves, skin).	{"feature": "whiteness", "chroma_w": 1.5}	0.324	0.434	0.353	0.898	0.235	0.038	25.68	0.23	Roughly neutral vs T01 (0.324 vs 0.327) but kept: chroma penalty is needed later to separate eggs from coloured bright surfaces once shape filtering is added.
T05	Add morphology (open 7, close 7) + hole filling to remove speckle and fill specular/shadow holes inside eggs.	{"feature": "whiteness", "chroma_w": 1.5, "open_k": 7, "close_k": 7, "fill_holes": true}	0.326	0.434	0.355	0.899	0.256	0.059	14.53	0.10	Kept: boundary F 0.236 -> 0.256, count noise down; IoU unchanged (0.326).
T06	Add shape filter on components (area, solidity, IoU with fitted ellipse, axis ratio): eggs are convex ellipses, background blobs are not.	{"feature": "whiteness", "chroma_w": 1.5, "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.8, "min_axis_ratio": 0.4}	0.244	0.279	0.331	0.284	0.234	0.403	1.01	0.09	Kept as a building block although IoU dropped to 0.244: instance precision jumped 0.03 -> 0.84, but single-Otsu blobs are usually egg+background merged, so the filter rejects most eggs (recall 0.28). Motivated T07.
T07	Replace single Otsu with a 12-level threshold sweep (40th-97th percentile) and pick the most egg-shaped components across levels (greedy NMS by ellipse score). One global threshold cannot fit egg-on-white and egg-on-dark scenes alike.	{"feature": "whiteness", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.8, "min_axis_ratio": 0.4}	0.407	0.507	0.587	0.619	0.362	0.493	0.75	0.21	Kept (big step): IoU 0.244 -> 0.407, Dice 0.508, instance F1 0.49. The sweep finds a level where each egg separates from its background.
T08	T07 + reject candidates not at least 15 grey-levels brighter than a 21px ring around them: many T07 false positives were flat elliptical background patches.	{"feature": "whiteness", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.8, "min_axis_ratio": 0.4, "min_contrast": 15}	0.395	0.487	0.550	0.602	0.357	0.507	0.75	0.32	Discarded: IoU 0.407 -> 0.395. The 15-level contrast gate also rejects real eggs on white backgrounds (low contrast).
T09	T07 but rank candidates by shape score x min(1, contrast/40) so bright-against-background ellipses win the NMS over flat ones.	{"feature": "whiteness", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.8, "min_axis_ratio": 0.4, "score": "shape_contrast"}	0.416	0.516	0.588	0.629	0.367	0.509	0.69	0.34	Kept (small): IoU 0.416 (+0.008), precision 0.588.
T10	T07 + keep only candidates scoring >= 0.85 x the best one in the image (most images have 1-3 similar eggs; weaker extra candidates were mostly false positives).	{"feature": "whiteness", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.8, "min_axis_ratio": 0.4, "rel_keep": 0.85}	0.440	0.533	0.638	0.614	0.385	0.523	0.59	0.29	Kept: IoU 0.440 (+0.032), precision 0.638, count MAE 0.59. Removes weaker secondary candidates.

Trial	What changed and why	Params (overrides vs previous / base)	IoU	Dice	Prec	Rec	BF	Inst F1	Count MAE	s/img	Decision
T11	T07 + area bonus 0.3*sqrt(area) in the ranking: nested sweep candidates (highlight inside egg) are both elliptical; prefer the larger whole egg.	{"feature": "whiteness", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.8, "min_axis_ratio": 0.4, "area_w": 0.3}	0.414	0.514	0.591	0.628	0.370	0.504	0.66	0.29	Alone only +0.007 IoU (0.414 vs T07 0.407); not adopted on its own, re-tested in combination (T14).
T12	Combine the two kept Stage-B changes (T09 contrast-aware score + T10 relative keep).	{"feature": "whiteness", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.8, "min_axis_ratio": 0.4, "score": "shape_contrast", "rel_keep": 0.85}	0.439	0.524	0.631	0.596	0.383	0.553	0.41	0.36	Kept as the combined base: IoU 0.439 (= T10 0.440 within noise) but instance F1 0.523 -> 0.553 and count MAE 0.59 -> 0.41.
T13	T12 + marker-controlled watershed refinement (fg = candidate eroded 0.5r, bg = outside candidate dilated 0.5r, flooded on the blurred colour image). Dev error maps showed the threshold keeps only the lit side of the egg; the shaded side is lost (low recall).	{"feature": "whiteness", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.8, "min_axis_ratio": 0.4, "score": "shape_contrast", "rel_keep": 0.85, "ws_refine": true}	0.492	0.561	0.638	0.639	0.461	0.577	0.41	0.84	Kept (big step): IoU 0.439 -> 0.492, boundary F 0.383 -> 0.461, recall 0.596 -> 0.639. Watershed recovers the shaded half of eggs.
T14	T13 + area bonus (T11) so the whole egg rather than its highlight seeds the refinement.	{"feature": "whiteness", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.8, "min_axis_ratio": 0.4, "score": "shape_contrast", "rel_keep": 0.85, "ws_refine": true, "area_w": 0.3}	0.475	0.548	0.616	0.643	0.454	0.573	0.34	1.00	Discarded: IoU 0.492 -> 0.476. With refinement the area bonus mostly promotes larger wrong regions.
T15	T12 + GrabCut (GMM colour model + graph cut, 3 iters, init from candidate) instead of watershed, as an alternative boundary refinement.	{"feature": "whiteness", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.8, "min_axis_ratio": 0.4, "score": "shape_contrast", "rel_keep": 0.85, "grabcut": true}	0.473	0.547	0.632	0.634	0.445	0.598	0.43	1.61	Discarded: GrabCut IoU 0.473 and BF 0.445 vs watershed 0.492 / 0.461, ~2x slower (1.61 vs 0.85 s/img); only instance F1 was higher (0.598 vs 0.577). GMM colour models confuse the grey shaded egg side with grey backgrounds.
T16	T13 + extra candidates from closed Canny contours (auto thresholds, dilate 3, fill holes). On white paper/boxes the egg has the same whiteness as the background, but its outline is still a visible edge.	{"feature": "whiteness", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.8, "min_axis_ratio": 0.4, "score": "shape_contrast", "rel_keep": 0.85, "ws_refine": true, "edge_cands": true}	0.496	0.564	0.638	0.643	0.465	0.593	0.38	0.95	Small gain (IoU 0.492 -> 0.496, BF 0.461 -> 0.465); re-tested in combination (T22).
T17	T13 with background-normalised whiteness (subtract 0.5 x Gaussian-blurred map, sigma 60): eggs on bright backgrounds and ash-covered eggs are locally brighter even if globally not.	{"feature": "whiteness_local", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.8, "min_axis_ratio": 0.4, "score": "shape_contrast", "rel_keep": 0.85, "ws_refine": true}	0.554	0.614	0.725	0.617	0.537	0.704	0.43	0.92	Kept (largest single gain): IoU 0.492 -> 0.554, precision 0.638 -> 0.725, BF 0.537. Bright paper/cloth no longer outranks the egg once the local background is subtracted.
T18	T13 with stricter ellipse-fit threshold 0.8 -> 0.85: several wrong picks were roughly elliptical paper/cloth regions.	{"feature": "whiteness", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.85, "min_axis_ratio": 0.4, "score": "shape_contrast", "rel_keep": 0.85, "ws_refine": true}	0.509	0.579	0.699	0.623	0.483	0.597	0.43	0.78	Kept: IoU 0.492 -> 0.509, precision 0.699 - stricter ellipse test removes elliptical-ish paper regions.
T19	T13 with stronger chroma penalty 1.5 -> 3.0 to push coloured backgrounds (skin, pink bowls, leaves) further down.	{"feature": "whiteness", "chroma_w": 3.0, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.8, "min_axis_ratio": 0.4, "score": "shape_contrast", "rel_keep": 0.85, "ws_refine": true}	0.481	0.552	0.658	0.609	0.466	0.603	0.45	0.71	Discarded: IoU 0.482 (< 0.492). Strong chroma penalty also darkens cream/pinkish eggs.
T20	T13 with a wider/denser sweep (20th-97th percentile, 16 levels) so darker / ash-covered eggs get a threshold level too.	{"feature": "whiteness", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.8, "min_axis_ratio": 0.4, "score": "shape_contrast", "rel_keep": 0.85, "ws_refine": true, "sweep_lo": 20, "sweep_levels": 16}	0.473	0.542	0.627	0.620	0.448	0.590	0.43	0.77	Discarded: IoU 0.473. Lower thresholds create more large merged blobs that win the ranking.

Trial	What changed and why	Params (overrides vs previous / base)	IoU	Dice	Prec	Rec	BF	Inst F1	Count MAE	s/img	Decision
T21	Combine T17 (local whiteness) + T18 (ellipse IoU >= 0.85).	{"feature": "whiteness_local", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.85, "min_axis_ratio": 0.4, "score": "shape_contrast", "rel_keep": 0.85, "ws_refine": true}	0.553	0.609	0.709	0.601	0.536	0.724	0.42	0.81	Kept as base (IoU 0.553, = T17 within noise); the two changes overlap in what they fix.
T22	T21 + Canny closed-contour candidates (T16 gave +0.004 alone).	{"feature": "whiteness_local", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.85, "min_axis_ratio": 0.4, "score": "shape_contrast", "rel_keep": 0.85, "ws_refine": true, "edge_cands": true}	0.560	0.613	0.710	0.607	0.541	0.736	0.39	0.82	Kept: best so far, IoU 0.560, BF 0.541. Edge candidates now help a little more because the refined, locally normalised pipeline ranks them correctly.
T23	T21 with stronger background subtraction (local_w 0.5 -> 0.8).	{"feature": "whiteness_local", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.85, "min_axis_ratio": 0.4, "score": "shape_contrast", "rel_keep": 0.85, "ws_refine": true, "local_w": 0.8}	0.540	0.587	0.676	0.586	0.515	0.696	0.39	1.16	Discarded: IoU 0.541. Over-subtraction flattens large eggs (their own brightness raises the blurred background).
T24	T21 with a smaller background scale (sigma 60 -> 30 px), closer to egg size (r ~ 40-120 px).	{"feature": "whiteness_local", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.85, "min_axis_ratio": 0.4, "score": "shape_contrast", "rel_keep": 0.85, "ws_refine": true, "local_sigma": 30}	0.507	0.551	0.608	0.575	0.490	0.695	0.51	0.97	Discarded: IoU 0.507. sigma 30 is smaller than many eggs, so the egg subtracts itself.
T25	T21 with a larger background scale (sigma 60 -> 120 px).	{"feature": "whiteness_local", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.85, "min_axis_ratio": 0.4, "score": "shape_contrast", "rel_keep": 0.85, "ws_refine": true, "local_sigma": 120}	0.547	0.599	0.699	0.594	0.522	0.675	0.41	0.86	Discarded: IoU 0.547 (< 0.553). sigma 60 kept.
T26	T22 but ellipse threshold back to 0.80 (T21 showed 0.85 adds nothing once local whiteness is used; looser may recover eggs with flattened outlines).	{"feature": "whiteness_local", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.8, "min_axis_ratio": 0.4, "score": "shape_contrast", "rel_keep": 0.85, "ws_refine": true, "edge_cands": true}	0.561	0.619	0.726	0.623	0.542	0.716	0.40	1.06	Marginal (IoU 0.561 vs 0.560); noted, re-tested on top of T28 (T29).
T27	T22 with looser relative keep 0.85 -> 0.7 to recover second/third eggs in multi-egg images (instance recall is the weak point).	{"feature": "whiteness_local", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.85, "min_axis_ratio": 0.4, "score": "shape_contrast", "rel_keep": 0.7, "ws_refine": true, "edge_cands": true}	0.542	0.599	0.690	0.609	0.527	0.717	0.43	1.15	Discarded: IoU 0.560 -> 0.542 and instance recall did not rise (0.669); the additional lower-scoring candidates were not the missed eggs.
T28	T22 with a wider watershed band (inner 0.35r, outer 0.7r): lets the refinement reach further into the shaded side of large eggs.	{"feature": "whiteness_local", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.85, "min_axis_ratio": 0.4, "score": "shape_contrast", "rel_keep": 0.85, "ws_refine": true, "edge_cands": true, "wr_in": 0.35, "wr_out": 0.7}	0.570	0.622	0.699	0.626	0.545	0.736	0.39	1.21	Kept: IoU 0.560 -> 0.570, recall 0.626, BF 0.545.
T29	T28 + looser ellipse threshold 0.8 (T26 was marginally better than T22 with it).	{"feature": "whiteness_local", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.8, "min_axis_ratio": 0.4, "score": "shape_contrast", "rel_keep": 0.85, "ws_refine": true, "edge_cands": true, "wr_in": 0.35, "wr_out": 0.7}	0.571	0.627	0.715	0.643	0.547	0.716	0.40	1.73	Kept candidate: IoU 0.570 -> 0.571, recall 0.643, BF 0.547 (within noise of T28).

Trial	What changed and why	Params (overrides vs previous / base)	IoU	Dice	Prec	Rec	BF	Inst F1	Count MAE	s/img	Decision
T30	T28 but replace each instance by its fitted ellipse before refinement: eggs are near-ellipses, so this should regularise ragged threshold outlines.	{"feature": "whiteness_local", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.85, "min_axis_ratio": 0.4, "score": "shape_contrast", "rel_keep": 0.85, "ws_refine": true, "edge_cands": true, "wr_in": 0.35, "wr_out": 0.7, "ellipse_fit": true}	0.573	0.625	0.700	0.629	0.547	0.736	0.39	1.40	Small gain over T28 (IoU 0.570 -> 0.573, BF 0.547) but recall fell to 0.629; superseded by T31.
T31	T28 + distance-transform watershed split of components that fail the shape test (touching eggs merge into one non-elliptical blob).	{"feature": "whiteness_local", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.85, "min_axis_ratio": 0.4, "score": "shape_contrast", "rel_keep": 0.85, "ws_refine": true, "edge_cands": true, "wr_in": 0.35, "wr_out": 0.7, "watershed": true, "ws_ratio": 0.6}	0.583	0.634	0.709	0.639	0.553	0.757	0.36	1.81	Kept, FINAL: best dev IoU 0.583 (+0.013 over T28), BF 0.553, instance F1 0.757, count MAE 0.36. Splitting merged non-elliptical blobs turns egg+background or egg+egg merges into valid candidates.
T32	T28 with an even wider refinement band (inner 0.25r, outer 0.9r).	{"feature": "whiteness_local", "chroma_w": 1.5, "thresh": "sweep", "open_k": 7, "close_k": 7, "fill_holes": true, "min_area": 0.004, "max_area": 0.6, "min_solidity": 0.9, "min_ell_iou": 0.85, "min_axis_ratio": 0.4, "score": "shape_contrast", "rel_keep": 0.85, "ws_refine": true, "edge_cands": true, "wr_in": 0.25, "wr_out": 0.9}	0.571	0.623	0.682	0.643	0.535	0.746	0.39	1.73	Discarded: IoU 0.571, no better than T28 (0.570) and below T31; precision fell 0.699 -> 0.682 because the 0.9r outer band lets the flood leak into the background.

How the pipeline evolved. The gray + Otsu baseline (T01) found most egg pixels but also whole bright backgrounds (precision 0.36). Shape filtering alone (T06) removed the background but also most eggs, because a single global threshold merges each egg with its surroundings. The multi-level threshold sweep (T07) fixed that by letting each egg separate from its background at some level (IoU 0.24 -> 0.41). Shape-and-contrast scoring with a relative keep threshold (T10, T12) cut false positives. Watershed edge refinement (T13) recovered the shaded half of the eggs (0.44 -> 0.49). Local background subtraction in the whiteness map (T17) was the largest single gain (0.49 -> 0.55), because bright paper and cloth no longer outrank the egg. Edge candidates, a wider refinement band and, finally, the watershed split of merged blobs (T31) took dev IoU to **0.583**. Several plausible ideas made things worse and were dropped: GrabCut instead of watershed (T15), a contrast gate (T08), a stronger chroma penalty (T19), lower thresholds (T20), smaller or larger background scales (T24, T25), and a looser keep threshold (T27).

Stage G (last checks around T28). T29 (looser ellipse test, 0.571), T30 (replace each egg by its fitted ellipse, 0.573), T31 (watershed split, 0.583) and T32 (wider refinement band, 0.571). T31 had the best dev mean IoU and also the best BF, instance F1 and count MAE among them, so it is the final configuration (`results/final_config.json`). The eval split was not used for any of these choices.

Task 3: Evaluation

Protocol

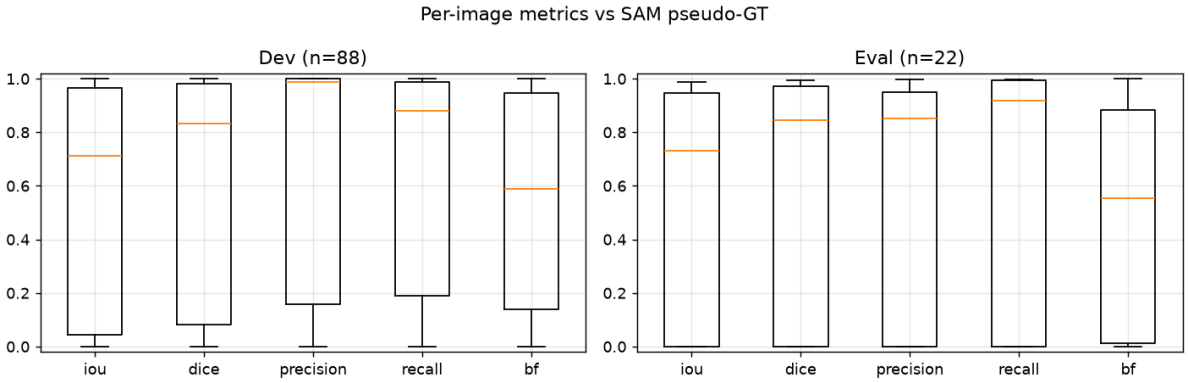
The final configuration (T31) was run once on the 22 held-out eval images, with the SAM masks from Task 1 as ground truth (`src/evaluate.py`). Metrics: - **IoU and Dice** of the egg/background mask, per image, then averaged. Pooled IoU and Dice sum the pixels over all images first, so large eggs weigh more. - **Pixel precision and recall:** the share of predicted pixels that are egg, and the share of egg pixels that were found. - **Boundary F:** precision and recall of boundary pixels matched within 9 px (1% of the image diagonal). - **Instance metrics:** predicted and SAM instances are matched with the Hungarian algorithm; a match needs IoU of at least 0.5. Count error is predicted eggs minus SAM eggs.

Results

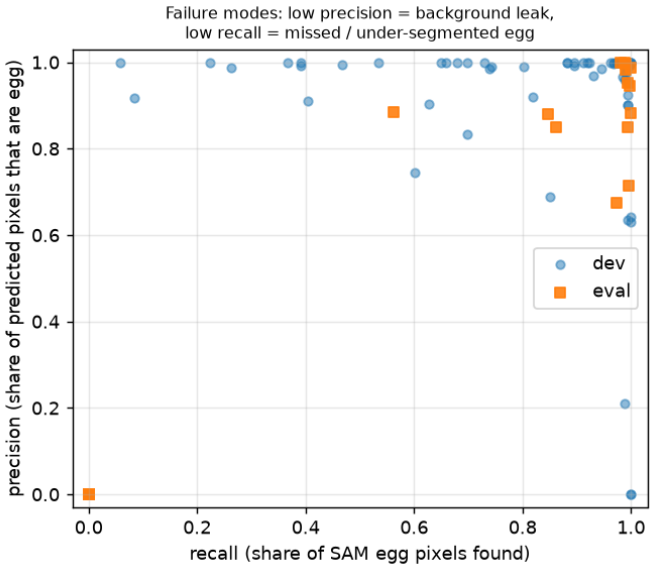
Metric	Dev	Eval (held-out)
Mean IoU (per image)	0.583	0.543
Mean Dice	0.634	0.581
Mean pixel precision	0.709	0.573
Mean pixel recall	0.639	0.599
Mean boundary F (tol 9 px)	0.553	0.502
Pooled IoU (all pixels)	0.601	0.604
Pooled Dice	0.751	0.753
Instance precision (IoU>=0.5)	0.832	0.536
Instance recall (IoU>=0.5)	0.694	0.652
Instance F1	0.757	0.588
Count MAE (eggs/image)	0.364	0.591
Exact count accuracy	0.693	0.545
Images	88	22

On the held-out split, mean IoU is **0.543** (dev 0.583), Dice 0.581, boundary F 0.502, and instance F1 0.588. Pooled IoU is the same on both splits (0.604 eval, 0.601 dev), so the drop in mean IoU comes from a few complete misses rather than worse masks overall.

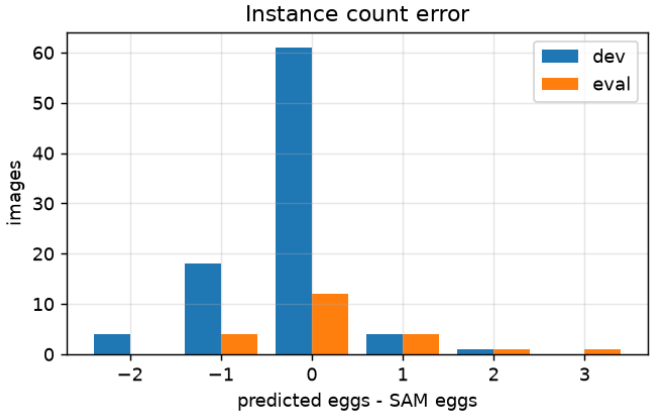
The per-image results are strongly bimodal. 14 of 22 eval images reach IoU 0.52-0.99 (median over all 22 images: 0.73), 9 of them at 0.85 or above with boundary F up to 1.00. 8 images are complete misses with IoU 0. Exact egg-count accuracy is 0.545: in 4 images the egg is missed altogether, and in 6 images there are extra detections (up to +3).



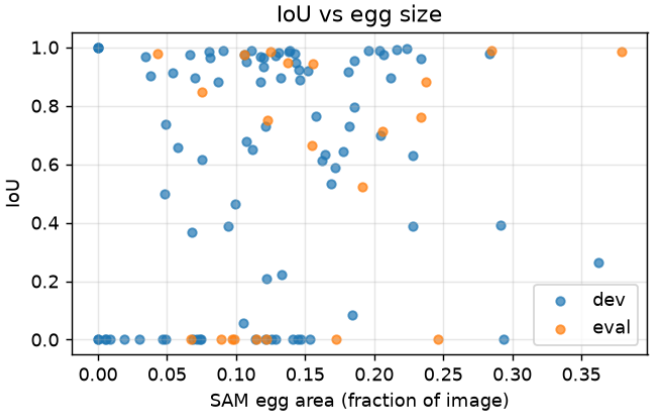
Per-image metric distributions on dev (88) and eval (22).



Pixel precision vs recall per image. Points at (0, 0) are complete misses; points with high recall and lower precision are background leaks.

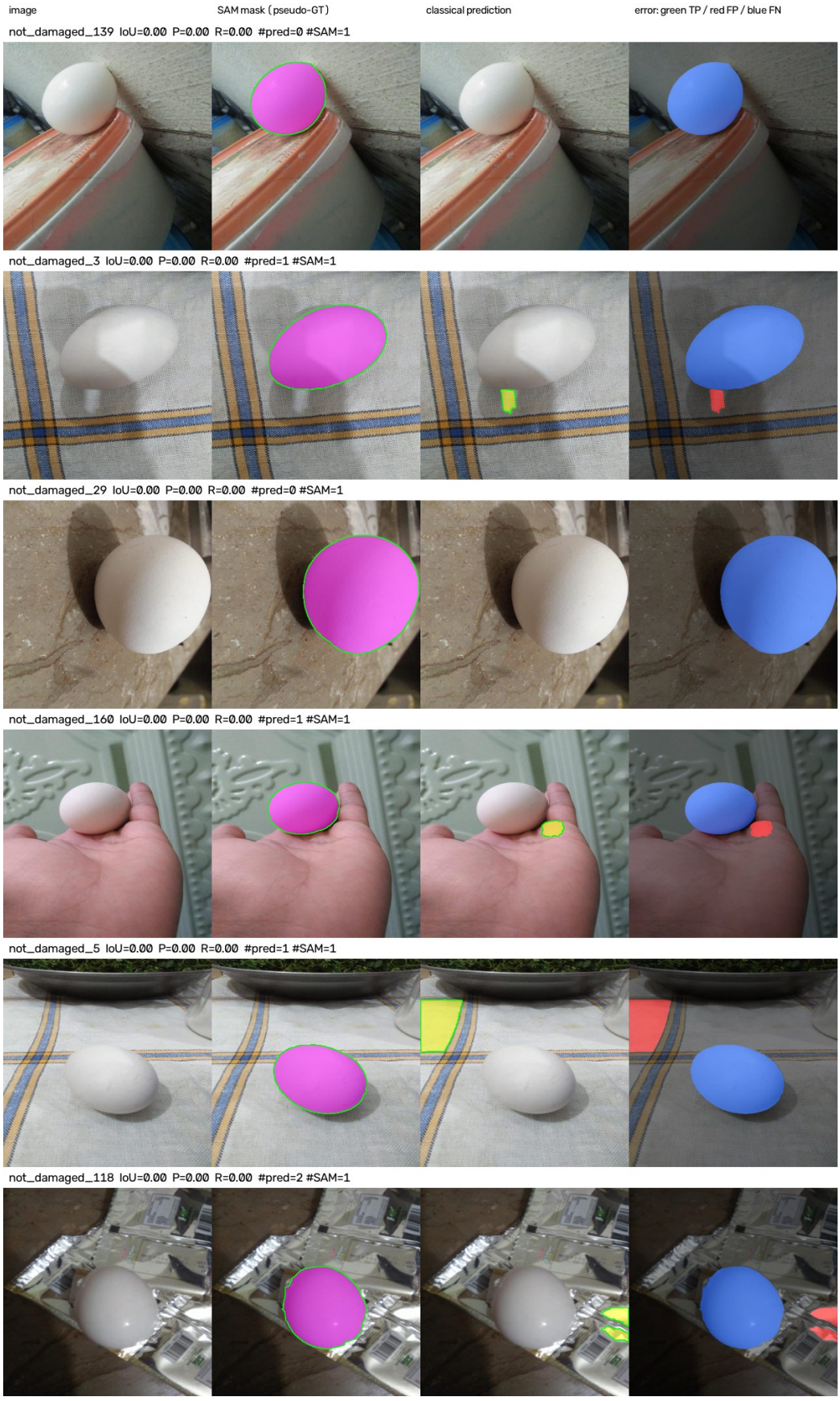


Instance count error (predicted - SAM) per image.



IoU vs egg size.

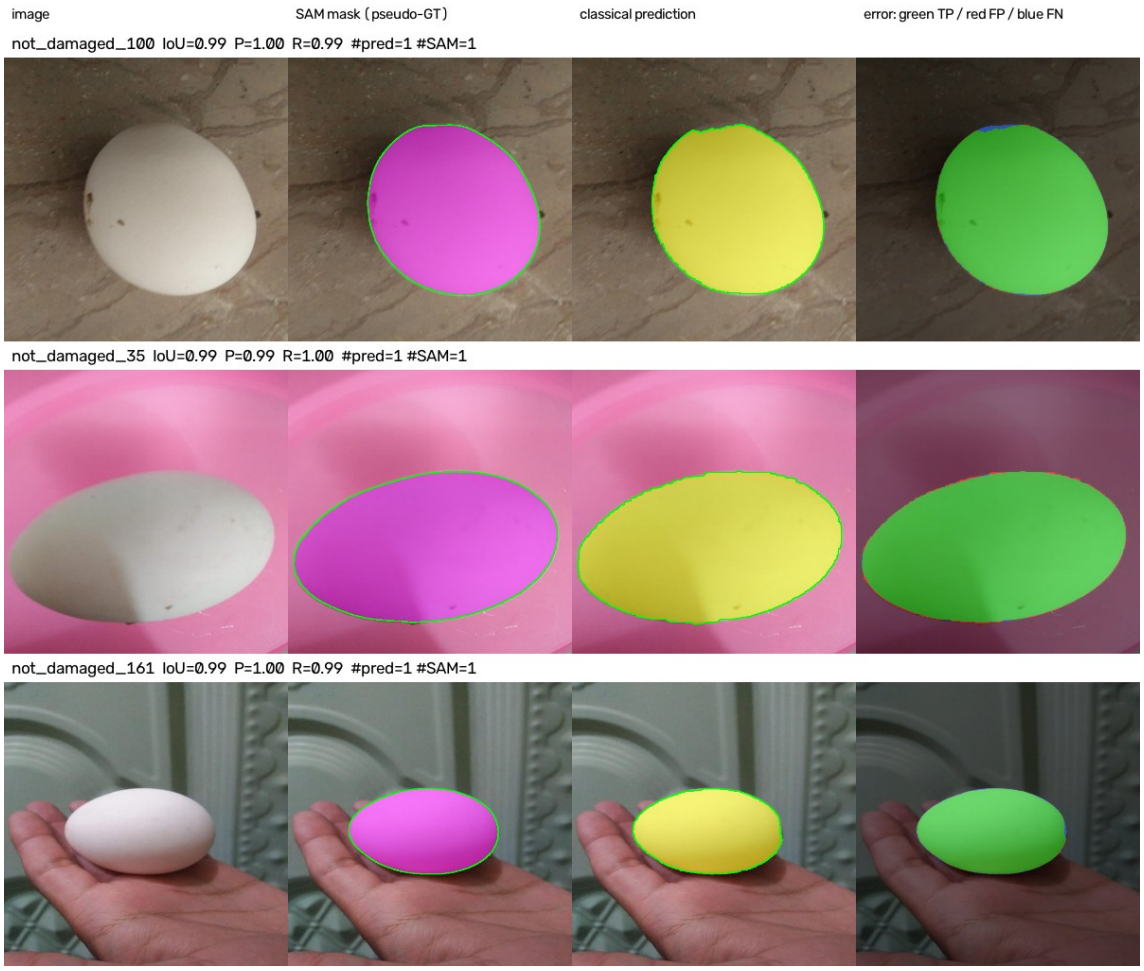
Failure analysis



The six worst eval images. Columns: image, SAM pseudo-GT, classical prediction, error map (green TP, red FP, blue FN).

All six worst images are true failures of the classical method; the SAM masks are correct in each of them. The common causes: - **White egg on a white or bright background** (cloth with stripes, a textured wall, a door, a white bowl). After local background subtraction the egg has almost no contrast against its surroundings, so no threshold level isolates it. The method then either finds nothing, or a small elliptical patch of background (a stripe, a fold) wins the ranking instead

of the egg. - **Egg on a hand:** the skin is bright with low chroma, and the egg merges with the palm in every threshold level. Only a small highlight on the hand passes the shape test. - **Specular foil:** bright, irregular reflections dominate the whiteness map. The egg itself is merged with the foil, while small elliptical reflections pass the shape test. - **Extra detections** in otherwise good images (count error +1 to +3), from round bright objects next to the egg.



The three best eval images for contrast: eggs with clear contrast to the background are segmented almost exactly.

What I would do next

- Rank candidates with edge evidence along the whole ellipse outline (gradient magnitude on the fitted ellipse), not only brightness contrast. That targets the white-on-white cases, where the outline is still visible.
- Use a skin-colour model to suppress hands, and a specularity test (saturated, high-gradient pixels) to suppress foil.
- Cleaner ground truth: SAM prompted with one click per egg, instead of the automatic generator plus a filter.
- With labels, a small learned model would be the obvious step. The classical pipeline is a reasonable baseline at about 1.8 s per image on CPU.

Task 4: Video tracking

Note on versions: the tracking video was produced with an **earlier version of the segmenter** than the final Task 2 configuration. It uses the global whiteness map, threshold sweep, morphology and ellipse-shape test (roughly the Stage A-B pipeline), with thresholds adapted to the video (`results/video_config.json`). It does not use local background subtraction, edge candidates, watershed splitting or watershed refinement. On this video (two white eggs on a dark table) that simpler segmenter was already sufficient, and I did not re-render the video with the final configuration.

Output video (public Google Drive link):
https://drive.google.com/file/d/10-KmLy1rw8uT85Oxb9gxMyuJAdiJGkqu/view?usp=sharing (local copy in the repo: `results/egg_tracking.mp4`, 640x360, 24 fps, 60 s, all 1440 frames processed).

The video (YouTube Y4JyV94y2_A, "Raw and Boiled Egg Spinning") shows two white eggs on a dark wooden table. A hand picks them up, spins them and puts them down. It also has title cards, white overlay text ("Plane surface", "Raw Egg"), table reflections under the eggs, a white-and-red checked shirt and a camera re-framing at about 30 s.

Segmenter adaptation

I used the same classical building blocks as Task 2 (whiteness map, threshold sweep, morphology, ellipse-shape scoring), in the earlier version described above. Only the acceptance thresholds changed, because this domain differs from the photo dataset. On a few sampled frames I printed the features of every candidate that passed the shape test: - Real eggs: ring contrast 139–168, mean whiteness 224–240. - Table reflections and finger tips: contrast 26–46, mean whiteness 93–131. - The green title card produced one huge elliptical blob (area 0.59 of the frame).

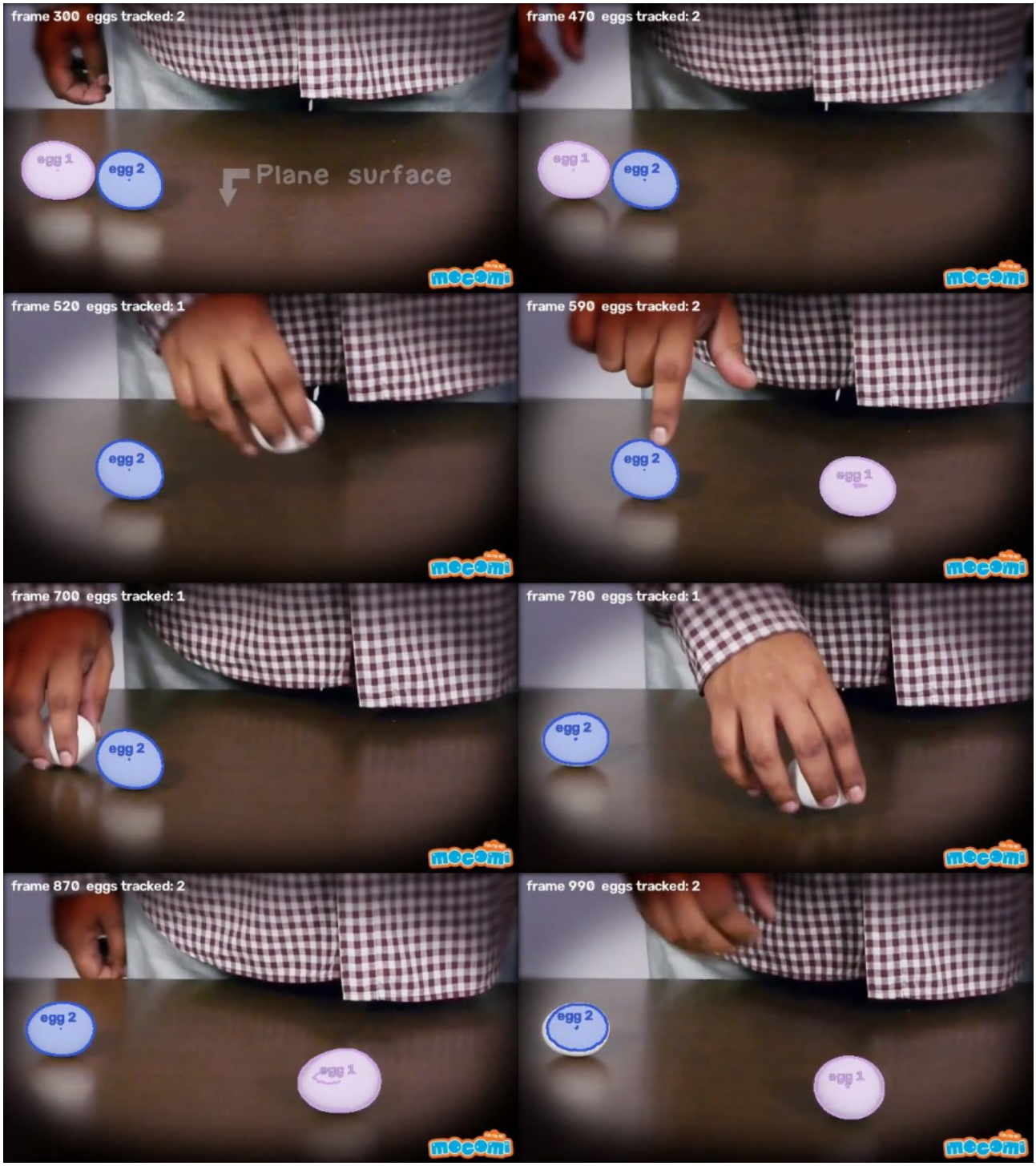
From those clearly separated clusters I set `min_contrast=70`, `min_mean_f=170`, `max_area=0.12`, `min_area=0.004` and `min_axis_ratio=0.5` (`results/video_config.json`). With these settings, reflections, text, fingers and the title card were no longer detected on the sampled frames. Frames are downscaled to 640x360, and the segmenter runs at about 0.15 s per frame on CPU.

Tracker

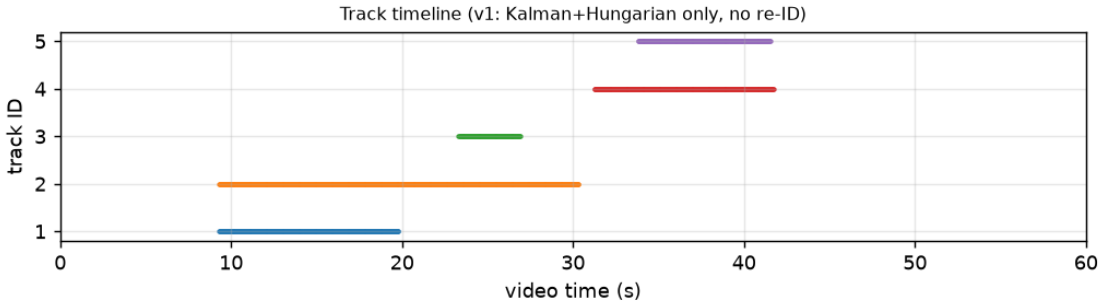
`src/tracker.py` works as follows: - Each track has a constant-velocity Kalman filter on the centroid. - Every frame, detections are assigned to tracks with the Hungarian algorithm on a cost of `0.5*(1 - IoU(det, track mask shifted by the Kalman-predicted motion)) + 0.5*(centroid distance / 2.5 egg radii)`. - A match is gated out if the distance exceeds 2.5 radii or the cost exceeds 0.85. - A track gets an ID after 3 hits (this suppresses one-frame false positives) and survives 12 missed frames (0.5 s) of occlusion or motion blur.

Version 1 (Kalman + Hungarian only) produced 5 IDs for 2 physical eggs (`results/tracks_v1_no_reid.csv`). The cause: when the hand lifts an egg, the egg is occluded for 2–4 s and put down somewhere else, so no motion model can bridge the gap. The camera re-framing at about 30 s also moves both eggs at once.

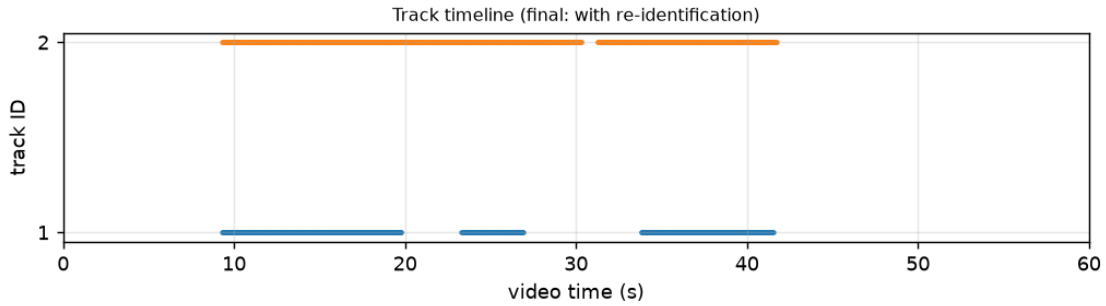
Version 2 (final) adds a lost-track pool for re-identification. A deleted confirmed track stays in the pool for 10 s, and a newly confirmed track inherits the ID of the lost track whose last position is nearest. This gave exactly 2 IDs over the whole video. I checked the result by eye on sampled frames: egg 1 is lifted, carried and put down to the right, and keeps ID 1; egg 2 keeps ID 2 through the re-framing.



Sampled frames from the tracking output (frames 300-990). Each egg keeps its ID through hand occlusion, a carry-and-place, and the camera re-framing.



Track timeline, v1 (no re-identification): 5 IDs for 2 eggs.



Track timeline, final: 2 persistent IDs. Gaps are frames where the egg is hidden in the hand or the frame is a title card.

Limitations of the tracking

- An egg held in the hand or spinning fast is often not detected: the fingers break its elliptical outline and motion blur lowers its contrast. The track simply pauses until the egg is visible again.
- Re-identification by nearest last position is a heuristic. The two eggs look identical, so if both disappear and reappear at the same time, their IDs could be swapped. Nothing in the pixels can resolve that. Fixes would be a hand tracker that "carries" the ID with the hand, or a small re-ID embedding.
- There is no quantitative tracking metric (MOTA/IDF1), because the video has no ground-truth tracks. The checks were visual plus the ID count (2 eggs in the video, 2 IDs).

AI tools disclosure

I used AI coding tools (Hark, an AI assistant) on this assignment: - **Implementation:** most of the Python code was written with Hark. That covers the SAM wrapper and egg filter, the split script, the classical pipeline with its configurable stages, the metrics, the tracker, the plotting and the PDF builder. - **Report drafting:** Hark drafted the text of this report and the README. - **Not AI-generated:** every number in the tables and figures comes from scripts in the repository, run on the data described here. The experiment log is the raw output of `run_trial.py` (`results/experiment_log.csv`), including the trials that made things worse.

I have reviewed the methods and can explain them and change them in the interview. Those are the Otsu and multi-level thresholding, the Lab whiteness map, the morphology, the ellipse-fit shape scoring, the distance-transform watershed, GrabCut, Hungarian assignment with a Kalman filter, and the boundary F-score.

Reproducing

See README.md. In short: - `scripts/00_download.sh` gets the data, video and weights. - `scripts/01_task1_sam.sh` runs SAM. - `scripts/02_task2_split_and_trials.sh` does the split and all the trials. - `scripts/03_task3_evaluate.sh` runs the evaluation. - `scripts/04_task4_track.sh` runs the tracking. - `src/make_report.py` builds this PDF.

Code: https://github.com/KKJ10000/nosh-egg-segmentation